2nd Test Plan Finance Tracker: Pencatatan Transaksi dan Insight

Panduan tes bertahap untuk pencatatan transaksi (income, expense, transfer, savings) dan generate insights, dimulai dari database yang baru dibuat. Financial setup hanya dimainkan secukupnya untuk menyiapkan data; tes detailnya ada di 1st Test Plan. Kirim setiap pesan ke bot satu per satu, lalu cek hasilnya di Telegram, di tab Executions n8n, dan di database. Diperbarui 30 Oktober 2026.

Urutan fasenya: buat database, smoke test, setup minimal, transaksi, tes negatif, insight, multi-intent, lalu cleanup. Transaksi memvalidasi bahwa akun, kategori budget, dan savings sudah ada, jadi setup harus lebih dulu. Kolom Harapan disusun dari membaca workflow, belum dijalankan, jadi hasil sebenarnya bisa berbeda. Selisihnya adalah temuan yang berguna. Isi kolom Hasil dengan OK atau GAGAL plus catatan singkat.

0. Persiapan (buat database dari nol)

1. Hentikan project dan hapus volume database lama: `docker compose down -v`. Perintah ini menghapus semua data database dan credential n8n, jadi export workflow dulu kalau perlu.
2. Nyalakan ulang dengan `bash start.sh`. `init.sql` hanya dijalankan saat volume baru dibuat, jadi tabel dan index terbentuk otomatis pada langkah ini.
3. Buka n8n, import workflow, buat ulang credential (Telegram, OpenAI, Postgres), pasang webhook lewat tunnel yang baru, lalu **Publish** dan pastikan **Active**. Isi *Restrict to User IDs* di Telegram Trigger.
4. Masuk ke database:

```
docker compose exec postgres sh -c 'psql -U $POSTGRES_USER -d $POSTGRES_DB'
```

5. Cek tabel dan index nama unik sudah ada:

```
\dt
SELECT indexname FROM pg_indexes WHERE tablename IN ('accounts','budgets','savings')
ORDER BY 1;
```

6. Pastikan semua tabel kosong:

```
SELECT (SELECT count(*) FROM accounts) AS accounts, (SELECT count(*) FROM budgets) AS
budgets, (SELECT count(*) FROM savings) AS savings, (SELECT count(*) FROM
transactions) AS transactions;
```

Tabel accounts, budgets, savings, dan transactions harus ada dan semua angka 0. Tabel transfers boleh ada kalau `init.sql` lamamu masih membuatnya, workflow tidak memakainya. Ketik `\q` untuk keluar dari psql (atau biarkan terbuka di jendela terminal lain).

Aturan main: kirim satu pesan, tunggu balasan, baru kirim yang berikutnya. Kalau ada yang aneh, buka tab Executions, klik eksekusi terbaru, dan lihat node mana yang merah atau keluarannya tidak sesuai.

Soal nama kategori: pakai nama Inggris (food, transport, entertainment) untuk budget. Validasi transaksi mencocokkan kategori dari parser dengan budget secara persis (tidak peka huruf besar-kecil). Kalau kategori dari parser transaksi tidak cocok dengan budget, kamu akan melihat peringatan invalid budget dan daftar kategori yang ada.

1. Smoke test

#	Pesan	Harapan	Hasil
1	`halo`	Balasan Unmatched Input (Input belum sesuai ketentuan)	
2	`get all data`	Dianggap insight, bukan Unmatched. Karena database kosong, jawabannya menjelaskan bahwa belum ada data	

2. Setup minimal (menyiapkan data)

Fase ini tidak sedetail 1st Test Plan: cukup memastikan akun, budget, dan savings terbentuk dengan angka yang dipakai fase transaksi. Kalau ada yang aneh, catat lalu lanjut, selama checkpoint di akhir terpenuhi.

#	Pesan	Harapan	Hasil
1	`buat akun BCA tipe bank saldo awal 5jt`	Akun dibuat. BCA, bank, 5000000	
2	`buat akun OVO tipe e-wallet saldo awal 500rb`	Akun dibuat. OVO, e-wallet, 500000	
3	`buat akun Cash tipe cash saldo awal 200rb`	Akun dibuat. Cash, cash, 200000	
4	`ubah nama akun Cash jadi Dompet`	Nama berubah jadi Dompet, saldo tetap 200000	
5	`tambah saldo akun BCA 100rb`	"Saldo bertambah", BCA Rp5.100.000	
6	`kurangi saldo akun BCA 100rb`	"Saldo berkurang", BCA kembali Rp5.000.000	
7	`hapus semua akun`	Warning: penghapusan massal tidak didukung. Tidak ada akun yang terhapus	
8	`buat budget food 500rb`	Budget dibuat. food 500000	
9	`buat budget transport 300rb`	Budget dibuat. transport 300000	
10	`buat budget entertainment 400rb`	Budget dibuat. entertainment 400000	
11	`tambah budget food 100rb`	food menjadi 600000	
12	`buat budget food 500rb`	Ditolak: budget "food" sudah ada, limit tetap 600000	

#	Pesan	Harapan	Hasil
13	`buat savings general target 10jt`	Savings dibuat. general, target 10000000	
14	`buat savings rumah target 200jt deadline 31 desember 2027`	Savings dibuat. rumah, target 200000000, deadline 2027-12-31	
15	`hapus semua savings`	Warning: penghapusan massal tidak didukung. Tidak ada savings yang terhapus	

16	Checkpoint:
	`SELECT account_name, account_type, balance FROM accounts ORDER BY account_id;`
	`SELECT category, limit_amount FROM budgets ORDER BY budget_id;`
	`SELECT savings_name, target_amount, current_amount FROM savings ORDER BY savings_id;`
		Akun: BCA bank 5000000, OVO e-wallet 500000, Dompet cash 200000. Budget: food 600000, transport 300000, entertainment 400000. Savings: general 10000000 (terkumpul 0), rumah 200000000 (terkumpul 0)	

3. Transaksi (income, expense, transfer, savings)

Sebutkan nama akunnya di setiap pesan. Validasi transaksi mengecek akun asal dan tujuan.

#	Pesan	Harapan	Hasil
1	`gaji masuk 10jt ke BCA`	Income, kategori salary. BCA menjadi 15000000	
2	`freelance 1jt ke OVO`	Income, kategori freelance. OVO menjadi 1500000	
3	`makan sushi 120rb pakai BCA`	Expense, kategori food. BCA 14880000. Balasan menampilkan pemakaian budget food sekitar 20 persen (120000 dari 600000)	
4	`grab ke kantor 50rb pakai OVO`	Expense, kategori transport. OVO 1450000. Pemakaian budget transport sekitar 17 persen	
5	`makan malam 300rb pakai BCA`	Food total 420000 (70 persen). BCA 14580000. Belum ada peringatan karena masih di bawah 80 persen	
6	`makan siang 200rb pakai BCA`	Food total 620000, melewati limit 600000. Peringatan budget terlampaui. BCA 14380000	

#	Pesan	Harapan	Hasil
7	`transfer 500rb dari BCA ke OVO`	BCA 13880000, OVO 1950000	
8	`tabung 1jt ke savings rumah dari BCA`	BCA 12880000, current_amount rumah 1000000	
9	`tabung 500rb ke savings general dari BCA`	BCA 12380000, current_amount general 500000	
10	`makan bakso 25rb pakai BCA tanggal 20 oktober 2026`	Expense food, transaction_date 2026-10-20. BCA 12355000. Food menghitung pengeluaran berdasarkan budget bulan Oktober.	
11	`makan 40rb dan parkir 10rb pakai BCA`	Dua-duanya tercatat dari BCA.	

Checkpoint:

```
SELECT account_name, balance FROM accounts ORDER BY account_id;
SELECT savings_name, current_amount FROM savings ORDER BY savings_id;
SELECT transaction_type, amount, category, from_account_id, to_account_id, savings_id, transaction_date FROM transactions ORDER BY transaction_id;
```

Harapan akhir: BCA 12355000, OVO 1950000, Dompet 200000. Savings rumah 1000000, general 500000.12 baris di tabel transactions.

4. Tes negatif transaksi (harus ditolak atau diperingatkan)

#	Pesan	Harapan	Hasil
1	`beli buku 80rb pakai BCA`	Invalid budget: tidak ada budget untuk kategori itu, daftar kategori ditampilkan. Saldo tidak berubah	
	`beli member gym 800rb pakai BCA`	Invalid budget: tidak ada budget untuk kategori itu, daftar kategori ditampilkan. Saldo tidak berubah	
2	`makan 50rb pakai Mandiri`	Invalid akun: Mandiri tidak ada, daftar akun ditampilkan	
3	`tabung 1jt ke savings mobil dari BCA`	Invalid savings: mobil tidak ada, daftar savings ditampilkan	
4	`makan 30rb`	Ditolak: akun pembayaran tidak disebut, jadi tidak ada akun yang ditebak (jangan sampai terpotong dari Dompet atau akun lain). Balasan: "akun pembayaran belum disebut" + daftar akun. Semua saldo tidak berubah dan tidak ada baris baru di transactions	

#	Pesan	Harapan	Hasil
5	`gaji masuk 1jt dan transfer 100rb ke OVO`	Ditolak karena input tidak lengkap (gaji masuk 1jt ke… dan transfer 100rb dari… ke OVO) --> missing value.	
6	`tabung 500rb dari BCA`	Tanpa nama savings: selalu masuk ke "general" (BCA berkurang 500.000, general bertambah 500.000), tidak pernah memilih tujuan lain sendiri. Balasan memuat catatan bahwa nama tidak disebut dan contoh cara menyebut tujuan. Kalau "general" tidak ada: ditolak "tabungan general tidak ditemukan"	
7	`tabung 500rb ke rumah dari BCA`	Savings disebut: masuk ke rumah, tanpa catatan default. BCA berkurang 500.000	
8	`gaji masuk 1jt`	Ditolak: akun penerima tidak disebut. Saldo tidak berubah	
9	`gaji masuk 5jt ke Mandiri`	Invalid akun tujuan: Mandiri tidak ada. Saldo tidak berubah	
10	`transfer 1jt dari BCA ke Mandiri`	Invalid akun tujuan: Mandiri tidak ada. BCA tidak berkurang	
11	`gaji masuk 10jt ke bc`	Invalid akun tujuan: bc tidak ada (typo). Saldo tidak berubah	

Checkpoint:

```
SELECT account_name, balance FROM accounts ORDER BY account_id;
SELECT savings_name, current_amount FROM savings ORDER BY savings_id;
SELECT transaction_type, amount, category, from_account_id, to_account_id, savings_id,
transaction_date FROM transactions ORDER BY transaction_id;
```

Harapan akhir: BCA 11805000, OVO 1950000, Dompet 200000. Savings rumah 1000000, general 500000. 13 baris di tabel transactions dengan tambahan transaksi savings sebesar 500000.

5. Insight

#	Pesan	Harapan	Hasil
1	`sisa uangku berapa?`	"Uang sisa" = total saldo semua akun kecuali kartu kredit dan loan (liquid_balance). Harus sama dengan jumlah saldo akun di database, tidak pernah minus karena rumus lama, dan tidak tercampur dengan uang yang belum dialokasikan.	

#	Pesan	Harapan	Hasil
2	`uang yang belum dialokasikan berapa?`	Dijawab sebagai insight: saldo dikurangi sisa budget bulan ini yang belum terpakai, ditampilkan beserta rinciannya (mis. saldo A − sisa budget B = C). Belanja yang masih di dalam budget tidak mengubah angkanya.	
3	`pengeluaran bulan ini berapa?`	Menghasilkan total pengeluaran bulan. Informasi Opsional: breakdown kategori jika relevan.	
4	`apa kategori yang paling boros?`	Menampilkan kategori dengan nominal pengeluaran tertinggi dan total nominal. Opsional: rekomendasi terhadap total pengeluaran.	
5	`budgetku aman ga sampai akhir bulan?`	Menyebut budget food sudah terlampaui (uang yang lebih dari limit dihitung Rp0), transport dan entertainment masih aman. Memberikan rekomendasi untuk menaikkan limit atau catat ke budget lain jika masih ada uang yang belum dialokasikan.	
6	`berapa persen budget transport yang terpakai?`	Menampilkan persentase budget yang terpakai, nominal budget, pengeluaran aktual, serta sisa budget.	
7	`tampilkan semua savings`	Nama ditulis persis seperti di database (rumah, general), tidak diterjemahkan atau diubah kapitalisasinya. Format: nama: Rp... dari target Rp... (persen), deadline ... (general: "belum ditentukan")	
8	`get all data`	Ringkasan akun, budget, savings, dan transaksi	
9	`kalau aku beli laptop 15jt, budgetku masih aman?`	Dijawab sebagai insight: tidak aman, karena 15000000 lebih besar dari total saldo atau uang yang belum dialokasikan. Tidak boleh mencatat transaksi baru	
10	`budget apa aja yang aku punya? enaknya kalo aku punya uang lebih, kira-kira aku bisa belanjain apa ya? pengen lebih merawat diri dan upgrade diri`	Menampilkan daftar budget yang dimiliki pengguna beserta rekomendasi pengeluaran yang relevan dan realistis.	

#	Pesan	Harapan	Hasil
11	`apakah kamu tau kartu kredit, e-wallet, atau bank apa yang belakangan ini sering memberi diskon untuk kategori makanan? kalo iya, apa ada rekomendasi pembuatan akun baru dengan diskon terbanyak atau tersering?`	AI tidak halusinasi, melainkan harus membatasi rekomendasi berdasarkan sumber data yang tersedia, yaitu data transaksi dan keuangan pengguna yang tercatat dalam database.	
12	`berapa pengeluaranku bulan ini per kategori?`	Menampilkan total pengeluaran secara keseluruhan dan per kategori.	

Note 1: Cek angka insight dengan SQL, jangan percaya begitu saja, contoh untuk 5.12:

```
SELECT category, SUM(amount) FROM transactions WHERE transaction_type = 'expenses'
GROUP BY category ORDER BY 2 DESC;
```

Note 2: Jumlah baris transactions tidak boleh bertambah (tetap 13).

```
SELECT count(*) FROM transactions;
```

6. Multi-intent dan variasi bahasa

#	Pesan	Harapan	Hasil
1	`gaji masuk 2jt ke BCA terus buat budget healthcare 300rb`	Dua cabang jalan: income tercatat (BCA 14315000) dan budget healthcare dibuat. Cek di Executions apakah tiap parser mengambil bagian yang benar (keduanya menerima seluruh pesan)	
2	`untung investasi saham 1M, masuk ke bca`	Pengeluaran dapat memahami kata 1M dan mengkategorikan jenis pemasukan selain salary.	
3	`hari ini dapet uang 1T dari hasil keuntungan jual tambang`	Parser transaksi saat ini hanya mengenali format rb, jt, k, M, sehingga perlu dilakukan pengecekan terhadap kemampuan pengenalan format serta klasifikasi jenis pengeluarannya.	
4	`i bought coffee 45k pakai OVO yesterday`	Pesan campuran English dan Indonesia. Kategori food dan nominal 45000.	
5	`my dad sent 2 million to my BCA account as pocket money two days ago`	Parser mengenali transaksi pemasukan sebesar 2000000 ke akun BCA, dengan jenis income uang saku dan tanggal transaksi dua hari sebelum tanggal hari ini.	

#	Pesan	Harapan	Hasil
6	`给 OVO 账户充值十万印尼盾`	Kata 充值 (chōngzhí) berarti top up / recharge, yang menunjukkan aktivitas penambahan saldo ke akun, bukan pengaturan finansial (financial_setup). Transaksi seharusnya tidak tercatat karena input tidak memberi informasi sumbernya (from_account).	
7	`Kemarin 收到工资 5jt masuk ke BCA, but i want to transfer 300rb nya ke OVO for shopping, terus beli groceries 150k pakai OVO. 请帮我设置每月 shopping budget 500k，并告诉我这个月的 spending breakdown`	Parser mengenali tiga transaksi (income, transfer, dan expense), mengategorikan groceries ke food, menetapkan budget shopping sebesar Rp500.000, serta menghasilkan insight tentang pengeluaran bulan ini.	

7. Update dan hapus (cleanup)

#	Pesan	Harapan	Hasil
1	`ubah budget healthcare jadi health`	Kategori berganti nama jadi health	
2	`hapus budget health`	Budget dihapus	
3	`hapus budget entertainment`	Budget dihapus	
4	`hapus savings general`	Savings dihapus. Transaksi dan activity log yang terhubung tetap ada, kolom savings_id menjadi NULL (sesuai ON DELETE SET NULL di init.sql)	
5	`hapus akun Dompet`	Akun dihapus	
6	`hapus budget shopping`	Ditolak: budget "shopping" tidak ada (satu pesan singkat dengan daftar budget yang ada)	

Checkpoint akhir:

```
SELECT category FROM budgets;
SELECT savings_name FROM savings;
SELECT account_name FROM accounts;
SELECT count(*) FROM transactions;
```

Harapan: budget food dan transport, savings rumah, akun BCA dan OVO. Jumlah transaksi tidak berkurang.

8. Mencatat temuan

4. Setiap pesan yang hasilnya tidak sesuai harapan: salin pesannya, intent atau hasil yang seharusnya, dan hasil sebenarnya.
5. Pesan-pesan itu dimasukkan ke daftar uji router dan parser, jadi tes ini sekaligus menjadi cikal bakal evaluasi untuk portfolio.
6. Perbaiki satu masalah per putaran (prompt atau node), lalu ulangi hanya pesan yang gagal ditambah beberapa pesan yang sebelumnya berhasil, supaya perbaikan tidak merusak yang sudah benar.
7. Kalau mau mengulang dari awal: `docker compose down -v`, `bash start.sh`, import workflow, buat ulang credential. Ingat bahwa `-v` menghapus semua data.